“Linux Process Monitoring and Control System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

R. VISHNU VARDHAN	**2520090166**
K. AMRUTHAVALLI	**2520090070**
N. DIVYA SREE	**2520090031**

Under the Esteemed Guidance of

Raghupathi M

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLGY



Declaration

The Project Report entitled **"Linux Process Monitoring and Control System"** is a record of Bonafide work of **R. VISHNU VARDHAN - 2520090166 , K. AMRUTHAVALLI - 2520090070, N.DIVYA SREE - 2520090031** submitted inpartial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

R. VISHNU VARDHAN	**2520090166**
K. AMRUTHAVALLI	**2520090070**
N. DIVYA SREE	**2520090031**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **"Linux Process Monitoring and Control System"** is a bonafide work done and submitted by **R. VISHNU VARDHAN - 2520090166 , K. AMRUTHAVALLI - 2520090070, N.DIVYA SREE - 2520090031** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Raghupathi M, and my guide Raghupathi M**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science and Information Technology*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

R. VISHNU VARDHAN 2520090166

K. AMRUTHAVALLI 2520090070

N. DIVYA SREE 2520090031

ABSTRACT

The Linux Process Monitoring and Control System is designed to provide a unified command-line environment for monitoring active processes and performing basic process-control operations in a Linux operating system. Process management is one of the fundamental responsibilities of an operating system, as it involves creating, executing, scheduling, monitoring, and terminating processes. In a multitasking environment, several processes execute simultaneously, making it important to understand their states, relationships, and execution details. Monitoring these processes helps users understand how system resources and running applications are managed by the operating system.

In a traditional Linux environment, users generally depend on different commands and system utilities to inspect process information and perform process-control operations. Although these utilities are useful, beginners may find it difficult to understand the relationship between process identification, process creation, parent–child relationships, and process execution. Therefore, this project aims to simplify the learning and demonstration of Linux process-management concepts by bringing essential monitoring and control functionalities into a single command-line interface.

The proposed system focuses on displaying important process-related information, including the Process ID (PID), Parent Process ID (PPID), and process state. These details help users identify running processes and understand the relationship between parent and child processes. The system utilizes the Linux /proc filesystem to access process-related information. It also incorporates important Linux system calls and interfaces such as fork(), exec(), wait(), and kill() to demonstrate process creation, execution, synchronization, and control, wherever supported by the implementation. The command-line interface accepts user commands, validates the provided input, performs the requested operations, and displays relevant process information or operation-status messages.

The project is developed using C/C++ programming languages in a Linux/Ubuntu environment, with the GCC compiler for compilation and execution. Development tools such as the Linux Terminal and Visual Studio Code support the development and testing process. The system is organized into essential functional modules, including process monitoring, process management, process control, system-interface handling, and output generation. Each module contributes to providing a structured approach to understanding and interacting with Linux processes.

The project also emphasizes the practical application of operating-system concepts through testing and observation. Important testing activities include verifying process identification, retrieving process states, observing parent–child process relationships, examining process creation and execution, testing supported process-control operations, and validating incorrect user inputs. These activities help evaluate the behavior of the system and provide practical exposure to Linux process-management mechanisms.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	**Implementation**		**30–45**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**62–67**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**79–83**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**84–86**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1: INTRODUCTION

1.1 Introduction

An Operating System (OS) is the primary software that manages computer hardware and software resources and provides an environment for executing applications. One of the most important responsibilities of an operating system is process management. A process is an instance of a program that is currently being executed. Modern operating systems handle multiple processes simultaneously, allowing users to perform different tasks efficiently.

Linux is a widely used open-source operating system that provides powerful facilities for process creation, execution, monitoring, and control. It maintains information about running processes, including their Process ID (PID), Parent Process ID (PPID), execution state, and other relevant details. These details help users understand how processes are created, managed, and terminated.

The Linux Process Monitoring and Control System is developed to provide a unified command-line environment for observing active processes and performing basic process-control operations. The system focuses on simplifying process-related activities by providing access to important process information through Linux interfaces. It also demonstrates essential operating-system mechanisms such as process creation, execution, parent–child relationships, and signal-based process control.

The project uses Linux facilities such as the /proc filesystem and relevant system calls to interact with process information and demonstrate process-management operations. Through this approach, users can gain practical knowledge of how Linux handles processes and how different process-management mechanisms work internally.

1.2 Background of the Project

Process management is an essential component of every multitasking operating system. Whenever a user launches an application or executes a command, the operating system creates and manages a corresponding process. Each process is assigned a unique Process ID (PID), while the Parent Process ID (PPID) helps identify the process responsible for creating it.

Linux provides several mechanisms to monitor and control processes. The /proc virtual filesystem exposes information about running processes, while system calls such as fork(), exec(), wait(), and kill() provide mechanisms for process creation, execution, synchronization, and control.

Understanding these mechanisms is important for students studying Operating Systems and Systems Programming. However, learning them only through theoretical explanations may not provide sufficient practical understanding.

This project addresses the learning requirement by bringing selected process-monitoring and control operations into a single command-line environment.

1.3 Problem Statement

In a Linux operating system, process-monitoring information and process-control operations are generally accessed through different commands and system interfaces. Beginners may find it difficult to understand how these operations are connected and how the operating system manages multiple running processes.

Identifying process details, understanding parent–child relationships, observing process states, and performing process-control operations require knowledge of different Linux mechanisms.

Therefore, there is a need for a simple and structured command-line system that demonstrates essential process-monitoring and control functionalities in one environment, making Linux process management easier to understand and explore.

1.4 Objectives of the Project

The primary objectives of the Linux Process Monitoring and Control System are:

1. To develop a command-line environment for monitoring Linux processes.
2. To display important process information, including PID, PPID, and process state.
3. To understand the relationship between parent and child processes.
4. To demonstrate process creation and execution using relevant Linux system calls.
5. To explore process-control mechanisms using Linux/POSIX signals.
6. To provide practical exposure to Linux process-management concepts.
7. To improve understanding of operating-system functionalities through practical implementation.

1.5 Scope of the Project

The scope of this project is focused on Linux process monitoring and basic process-control operations through a command-line interface.

The system covers the following areas:

- **Process Monitoring:** Retrieving and displaying process information such as PID, PPID, and process state.
- **Process Identification:** Identifying individual processes using their unique process identifiers.
- **Process Creation:** Understanding how new processes are created using the fork() system call.
- **Process Execution:** Exploring program execution using the exec() family of functions.
- **Process Synchronization:** Understanding parent–child process coordination using wait().
- **Process Control:** Demonstrating supported process-control operations using kill() and Linux signals.
- **Command-Line Interaction:** Accepting user commands, validating inputs, and displaying process information and operation status.
- The project primarily focuses on educational demonstrations of Linux process-management concepts. Advanced graphical monitoring, continuous resource analytics, and automated alerts are considered possible future enhancements.

1.6 Importance of the Application

The Linux Process Monitoring and Control System provides an opportunity to understand the internal working of a Linux operating system through practical activities.

It helps students understand how processes are identified, created, executed, monitored, and controlled. By working with Linux system interfaces and process-related information, students can connect theoretical concepts with actual operating-system behavior.

The command-line approach also provides a lightweight environment for demonstrating process-management mechanisms without requiring a complex graphical interface.

1.7 Target Users / Audience

The primary target users of this project are:

- Computer Science and Engineering students.
- Students studying Operating Systems and Systems Programming.
- Beginners learning Linux process management.
- Users interested in understanding Linux system calls and process execution.
- Learners exploring basic process-monitoring and control mechanisms.

CHAPTER -2: OS CONCEPTS AND BACKGROUND STUDY

2.1 Introduction to Process Management

Process management is one of the major functions of an operating system. It is responsible for creating, scheduling, executing, monitoring, and terminating processes. A process represents a program that is currently being executed by the computer.

In a multitasking operating system such as Linux, multiple processes can exist and execute concurrently. The operating system manages these processes and allocates the necessary resources, including CPU time, memory, and input/output resources.

Every process contains information required by the operating system to manage its execution. This includes process identification details, execution state, memory information, and relationships with other processes.

The Linux Process Monitoring and Control System uses these concepts to provide information about active processes and demonstrate basic process-management operations.

2.2 Process and Program Concepts

A program and a process are two different concepts in operating systems.

A program is a passive collection of instructions stored in a file on a storage device. It does not perform any operation until it is executed.

A process is an active instance of a program that is currently running. When a user executes a program, the operating system creates a process and allocates the resources required for its execution.

The major differences are:

Program	Process
It is a passive set of instructions.	It is an active executing instance.
Stored on a storage device.	Managed in main memory and by the OS.
Does not have an execution state.	Has a specific execution state.
Does not require a PID.	Identified using a PID.
Remains stored until modified or deleted.	Exists during its execution lifecycle.

For example, a text editor installed on a computer is a program. When the user opens it, the operating system creates a process to execute that program.

2.3 Process Identification – PID and PPID

Every process in Linux is assigned a unique Process ID (PID), which is used by the operating system to identify and manage that process.

The Parent Process ID (PPID) represents the PID of the process that created the current process. Linux maintains parent–child relationships between processes, allowing the operating system to track how processes are created.

The two important identifiers are:

1. Process ID (PID)

- Identifies an individual process.
- Used to distinguish one running process from another.
- Can be used when requesting process information or performing supported control operations.

2. Parent Process ID (PPID)

- Identifies the parent process.
- Helps understand the relationship between processes.
- Allows users to observe how one process creates another.

For example, consider a parent process with PID 1200 that creates a child process with PID 1250.

Process	PID	PPID
Parent Process	1200	900
Child Process	1250	1200

In this example, the PPID of the child process is 1200, which matches the PID of its parent process.

The Linux Process Monitoring and Control System uses PID and PPID information to help users identify processes and understand their relationships.

2.4 Process States

A process does not remain in the same execution condition throughout its lifetime. It moves between different states depending on CPU availability, resource requirements, and execution progress.

The commonly studied process states are:

1. New State

A process enters the new state when it is being created by the operating system.

2. Ready State

The process is prepared for execution and is waiting for CPU allocation.

3. Running State

The process is currently executing instructions using the CPU.

4. Waiting / Blocked State

The process is waiting for an event or resource, such as the completion of an input/output operation.

5. Terminated State

The process has completed execution or has been stopped.

These states help explain how the operating system manages multiple processes.

The following diagram represents the general process-state transition model:

NEW → READY → RUNNING → TERMINATED

A running process may also move back to the ready state or enter a waiting state depending on scheduling decisions and resource requirements.

Linux uses its own internal process-state representations. The project focuses on retrieving and displaying the process state available through Linux process information interfaces.

2.5 Process Control Block (PCB)

A Process Control Block (PCB) is a data structure maintained by the operating system to store important information about a process.

The PCB allows the operating system to track and manage individual processes during execution.

Important information maintained for a process includes:

- **Process ID:** Unique identifier assigned to the process.
- **Process State:** Current execution condition of the process.
- **Program Counter:** Address of the next instruction to execute.
- **CPU Registers:** Information required to resume process execution.
- **Scheduling Information:** Details used by the CPU scheduler.
- **Memory Information:** Information related to the process memory.
- **I/O Information:** Details about input/output resources used by the process.

The operating system uses process-related information to manage execution, perform context switching, and maintain process relationships.

In Linux, process information is maintained through kernel data structures, including the task structure. The /proc filesystem exposes selected process information to user-space applications.

2.6 Process Creation and Execution

Process creation is an important operation performed by an operating system. In Linux, a process can create another process using the fork() system call.

The newly created process is known as the child process, while the process that creates it is called the parent process.

The child process receives a separate process identity and initially inherits relevant execution context from its parent.

The following system calls are important in process creation and execution:

1. fork()

The fork() system call creates a new child process. It returns different values to the parent and child processes, allowing them to identify their execution paths.

2. exec()

The exec() family of functions replaces the current process image with a new program. It is commonly used after fork() when a child process needs to execute another application.

3. wait()

The wait() system call allows a parent process to wait for a state change in one of its child processes, commonly its termination.

These system calls help demonstrate how Linux creates, executes, and manages processes.

2.7 Linux /proc Filesystem

The /proc filesystem is a virtual filesystem provided by the Linux operating system. Unlike a normal filesystem, it does not primarily store regular files on a physical storage device. Instead, it provides access to information maintained by the Linux kernel.

It contains information about running processes and other system-level details.

Each process generally has a corresponding directory under /proc, identified by its numeric PID.

For example:

- /proc/1200 contains information associated with process ID 1200.
- /proc/1250 contains information associated with process ID 1250.
- /proc/1200/status provides status information about process 1200.

Some important entries include:

/proc/[PID]/status: Provides information such as process identifiers and process state.

/proc/[PID]/stat: Contains detailed process statistics, including process state and parent process ID.

/proc/[PID]/cmdline: Provides the command-line arguments associated with a process.

The project uses the Linux /proc filesystem as an interface for retrieving relevant process information, including PID, PPID, and process state.

2.8 Process Control and Signals

Process control refers to operations that influence or manage the execution of processes.

Linux provides signals as a mechanism for communicating with processes and requesting specific actions.

A signal is a notification delivered to a process to indicate that a particular event has occurred or that an action is requested.

Some commonly used Linux signals are:

Signal	Description
SIGTERM	Requests a process to terminate gracefully.
SIGKILL	Forces a process to terminate immediately.
SIGSTOP	Stops the execution of a process.
SIGCONT	Resumes a stopped process.
SIGINT	Interrupts a process, commonly through Ctrl+C.

The kill() system call is used to send signals to processes. Despite its name, it can send different signals and is not limited to terminating processes.

The project explores process-control operations using Linux/POSIX signals, subject to the operations supported by its implementation and the permissions of the executing user.

2.9 Existing System and Proposed Approach

In a traditional Linux environment, users can access process information and control operations through different commands and system utilities.

Commands such as ps and top are commonly used to inspect running processes. Other Linux interfaces provide facilities for process creation, execution, synchronization, and control.

Although these tools are useful, beginners may need to understand multiple commands and interfaces separately.

The proposed Linux Process Monitoring and Control System aims to organize selected monitoring and process-control operations through a unified command-line environment.

The system accepts user commands, validates the input, accesses relevant Linux process information or system interfaces, performs supported operations, and displays the corresponding results.

This approach helps students understand the connection between different process-management concepts through practical demonstrations.

2.10 Advantages of the Proposed System

1. The proposed system provides the following advantages:
2. Provides a unified command-line environment for selected process-monitoring and control operations.
3. Helps users identify processes using PID and PPID.
4. Improves understanding of parent–child process relationships.
5. Demonstrates Linux process creation and execution mechanisms.
6. Provides practical exposure to Linux system calls and signals.
7. Helps connect theoretical operating-system concepts with practical implementation.
8. Uses Linux and GCC development tools, which are widely available and open source.

2.11 Summary

This chapter discussed the fundamental operating-system concepts related to the Linux Process Monitoring and Control System. It covered process management, the difference between programs and processes, process identification, process states, Process Control Blocks, process creation, Linux system calls, the /proc filesystem, and signal-based process control.

These concepts form the theoretical foundation of the proposed project and help explain how Linux manages running processes. The next chapter discusses the hardware and software requirements necessary for developing and executing the system.

CHAPTER -3: SYSTEM REQUIREMENTS

3.1 Introduction

System requirements define the hardware and software resources necessary for developing, compiling, executing, and testing an application. Every software project requires a suitable computing environment to ensure proper functionality and reliable execution.

The Linux Process Monitoring and Control System requires a Linux-based environment because it interacts with operating-system process information and Linux system interfaces. The project uses C/C++ programming languages and the GCC compiler for development and execution.

The system requirements are divided into hardware requirements and software requirements. These requirements help establish the environment needed to monitor processes, retrieve process information, and perform supported process-control operations.

3.2 Hardware Requirements

Hardware requirements specify the physical components necessary to develop and execute the project.

The following hardware configuration is suitable for running the Linux Process Monitoring and Control System:

Component	Recommended Requirement
Processor	Intel Core i3 or equivalent and above
RAM	Minimum 4 GB
Storage	10 GB available space recommended
Display	Standard monitor or laptop display
Keyboard	Standard keyboard
Mouse	Standard pointing device
System Architecture	64-bit processor

A computer with these specifications is sufficient for running a Linux environment, compiling C/C++ programs, and performing basic process-monitoring operations.

Since the project primarily uses command-line operations and Linux system interfaces, it does not require specialized hardware or high-performance computing resources.

3.3 Software Requirements

Software requirements specify the operating system, programming languages, compilers, and development tools required for implementing the project.

The following software components are used or required:

Software	Purpose
Linux / Ubuntu	Operating system environment
C / C++	Programming languages
GCC Compiler	Compilation of source code
Linux Terminal	Executing commands and programs
Visual Studio Code	Source-code editing
Linux /proc Filesystem	Retrieving process information
POSIX / Linux APIs	Process creation and control
Git and GitHub	Source-code version control, if used

These software components provide the necessary environment for developing and executing the project.

3.4 Operating System Environment – Ubuntu / Linux

Linux is an open-source operating system widely used for software development, system programming, and server management. It provides several built-in facilities for managing processes and interacting with system resources.

The proposed project uses a Linux environment because its functionality depends on Linux-specific process information and system interfaces.

Ubuntu is considered the target operating system environment for this project. It provides a user-friendly Linux distribution with access to the GCC compiler, terminal utilities, system libraries, and development tools.

The Linux environment supports important process-management operations through the following interfaces:

1. Process Information

Linux provides the /proc virtual filesystem, which exposes information about running processes. It allows applications to retrieve process-related details such as PID, PPID, and process state.

2. Process Creation

Linux provides the `fork()` system call, which allows a running process to create a child process.

3. Process Execution

The `exec()` family of functions allows a process to replace its current execution image with another program.

4. Process Synchronization

The `wait()` system call allows a parent process to wait for changes in the state of its child processes.

5. Process Control

Linux supports signals and the `kill()` system call for sending signals to processes.

These facilities provide the operating-system environment required for demonstrating the project's process-monitoring and control functionalities.

3.5 Development Tools and Compilers

Development tools and compilers play an important role in writing, compiling, debugging, and executing the project.

The following tools are associated with the project:

3.5.1 GCC Compiler

The GNU Compiler Collection (GCC) is used to compile C and C++ source code into executable programs. It is widely used in Linux-based development environments.

GCC provides compiler diagnostics that help identify syntax errors, compilation issues, and other problems in source code.

3.5.2 Linux Terminal

The Linux Terminal provides a command-line interface for interacting with the operating system.

It is used to execute Linux commands, compile source files, run the application, and inspect process information.

3.5.3 Visual Studio Code

Visual Studio Code is a source-code editor that can be used to write and manage C/C++ programs.

It provides features such as syntax highlighting, code navigation, extensions, and integrated terminal support, making the development process easier.

3.5.4 Git and GitHub

Git is a version-control system used to track changes in source code. GitHub provides an online platform for hosting and managing Git repositories.

These tools can support collaboration among team members, maintain project versions, and preserve source-code history.

3.6 System Configuration

The project requires a properly configured Linux development environment with the necessary compiler and system utilities.

The following configuration steps are generally required:

1. Install or configure a Linux/Ubuntu environment.
2. Install the GCC compiler.
3. Configure the development environment.
4. Create the project source-code files.
5. Compile the C/C++ source code.
6. Execute the compiled application through the Linux Terminal.
7. Verify process information and supported process-control operations.

The GCC compiler can be installed in Ubuntu using the following command:

```
sudo apt update

sudo apt install build-essential
```

The installed compiler can be checked using:

```
gcc --version
```

For C++ development, the compiler version can be checked using:

```
g++ --version
```

These commands help verify that the required compilation environment is available.

3.7 Summary

This chapter described the hardware and software requirements necessary for developing and executing the Linux Process Monitoring and Control System. It also explained the Linux/Ubuntu operating environment, GCC compiler, Linux Terminal, Visual Studio Code, and other development tools.

The project uses standard computing hardware and freely available Linux development tools, making it suitable for an academic environment. The next chapter explains the technologies and tools used in the implementation of the system.

CHAPTER -4: TECHNOLOGY AND TOOLS USED

4.1 Introduction

The Linux Process Monitoring and Control System uses programming languages, Linux utilities, system interfaces, and development tools to perform process-related operations. Each technology plays a specific role in developing, compiling, executing, and testing the application. This chapter explains the technologies used in the project and their importance.

4.2 C/C++ Programming Language

C/C++ is used for system-level programming because it provides access to operating system features and supports efficient execution. These languages allow programmers to work with process-related functions and interact with Linux system interfaces.

In this project, C/C++ serves as the primary programming technology for implementing process monitoring and control operations. It supports structured programming, function-based development, and interaction with Linux system calls.

4.3 GCC Compiler

The GNU Compiler Collection (GCC) is a compiler used to convert C and C++ source code into executable programs.

GCC is useful in this project because it allows the developed source code to be compiled and executed in a Linux environment. It also helps identify syntax errors and compilation issues during development.

4.4 Linux Operating System

Linux is an open-source operating system that provides process management facilities and system-level programming interfaces.

The project is designed to operate in a Linux environment, such as Ubuntu. Linux manages processes using process identifiers, maintains process information, and provides mechanisms for process creation, execution, and control.

4.5 Linux /proc Filesystem

The `/proc` filesystem is a virtual filesystem provided by Linux. It exposes information about running processes and various kernel parameters.

Each process generally has a directory under `/proc` named using its Process ID (PID). Files such as `status` contain information about a process, including its name, process identifiers, and state.

The project can use information exposed through `/proc` to inspect process details. Unlike an ordinary disk-based filesystem, `/proc` provides information dynamically through the Linux kernel.

4.6 Linux System Calls

System calls provide an interface through which programs request services from the operating system. Linux provides several system calls related to process management.

The important process-related functions associated with this project include:

fork(): Creates a new process by duplicating the calling process. The newly created process is called the child process.

exec(): Replaces the current process image with a new program. It is commonly used after process creation when a child process needs to execute another program.

wait(): Allows a parent process to wait for a child process to change state, commonly to complete execution.

kill(): Sends a specified signal to a process identified by its PID. Depending on the signal, this can request termination or perform another supported action.

These functions provide the basic mechanisms required for process creation, execution, synchronization, and control. Their actual usage in the implementation should correspond to the project's source code.

4.7 POSIX and Linux Signals

Signals are software notifications used by the operating system or processes to communicate events. They provide a mechanism for sending notifications to processes.

Examples include:

- **SIGTERM:** Requests a process to terminate.
- **SIGKILL:** Forces a process to terminate and cannot be caught or ignored.
- **SIGSTOP:** Stops a process.
- **SIGCONT:** Resumes a stopped process.

Signals are relevant to process control because they allow processes to receive and respond to operating-system notifications.

4.8 Linux Terminal

The Linux Terminal provides a command-line interface for interacting with the operating system. It allows users to execute commands, compile programs, run applications, and inspect system information.

In this project, the terminal is used as the primary environment for compiling, executing, and testing the process monitoring application.

4.9 Visual Studio Code

Visual Studio Code is a source-code editor that supports programming through syntax highlighting, code navigation, extensions, and debugging features.

It can be used to write and maintain the project's C/C++ source files. Its lightweight interface and development extensions make it suitable for Linux-based programming.

4.10 Git and GitHub

Git is a version-control system used to track changes in source code. GitHub is a platform that hosts Git repositories and supports collaboration.

When used in the project, these tools help maintain code versions, preserve development history, and share source files among team members.

4.11 Summary

This chapter described the technologies and tools associated with the Linux Process Monitoring and Control System. C/C++ supports system-level programming, GCC compiles the source code, and Linux provides the process-management environment. The `/proc` filesystem and process-related system calls provide access to process information and control mechanisms. The Linux Terminal and Visual Studio Code support development and testing, while Git and GitHub can support source-code management and collaboration.

CHAPTER 5: SYSTEM DESIGN AND ARCHITECTURE

5.1 Introduction

System design describes the structure, components, and working principles of a software application. It explains how different modules interact to achieve the objectives of the system.

The Linux Process Monitoring and Control System is designed as a command-line application that allows users to inspect Linux processes and perform supported process-management operations. The proposed architecture focuses on simplicity, modularity, and interaction with Linux operating system interfaces.

5.2 System Architecture

The system follows a command-line-based architecture in which the user interacts with the application through the Linux Terminal.

The application receives a user request, identifies the requested operation, interacts with the appropriate Linux interface, and displays the result.

Proposed System Architecture:

User → Linux Terminal → Process Monitoring and Control Application → Linux System Interfaces → Linux Kernel / Process Information

The results are returned to the application and displayed to the user through the terminal.

The major components are:

1. **User Interface:** Accepts user commands and displays information.
2. **Command Processing Module:** Identifies and validates requested operations.
3. **Process Monitoring Module:** Retrieves process-related information.
4. **Process Control Module:** Handles supported process-control requests.
5. **Linux System Interface:** Provides access to process information and operating system services.

5.3 Module Description

The proposed system is divided into logical modules to simplify development and maintenance.

5.3.1 User Interface Module

The User Interface Module provides communication between the user and the application.

It is responsible for accepting input through the terminal, presenting available operations, and displaying process details or operation results.

A command-line interface is suitable for this project because it requires fewer system resources and provides direct interaction with Linux utilities and system interfaces.

5.3.2 Process Monitoring Module

The Process Monitoring Module is responsible for obtaining information about Linux processes.

The information may include:

- Process ID (PID)
- Parent Process ID (PPID)
- Process name
- Process state

Linux exposes process-related information through the `/proc` filesystem. The monitoring module can use this interface to retrieve and present relevant process details.

5.3.3 Process Control Module

The Process Control Module manages supported operations involving process execution and control.

Linux provides interfaces such as `fork()`, `exec()`, `wait()`, and `kill()` for process-related operations.

The module is designed around these operating system mechanisms. The exact operations available to the user depend on the functions implemented in the final source code.

5.3.4 Input Validation and Error Handling Module

Input validation helps ensure that user requests are processed correctly.

The system should check whether the requested operation is supported and whether the required input values are valid.

For example, a process-related operation may fail when the specified PID does not exist or when the user does not have sufficient permissions.

Appropriate error messages help users understand why an operation could not be completed.

5.3.5 Output Display Module

The Output Display Module presents information in a readable terminal format.

It displays process identifiers, process states, operation results, and error messages.

Clear output formatting improves usability and helps users understand the results of process-monitoring and control operations.

5.4 Data Flow of the System

The proposed data flow describes how information moves between the user, application, and operating system.

Step 1: The user starts the application in the Linux Terminal.

Step 2: The application receives a command or operation selection.

Step 3: The input is checked to identify the requested operation.

Step 4: The appropriate monitoring or control module processes the request.

Step 5: The application accesses the required Linux process information or system interface.

Step 6: The result is returned to the application.

Step 7: The application displays the result or an appropriate error message.

This flow provides a structured approach to handling process-related requests.

5.5 Process Monitoring Design

The process monitoring component is designed to retrieve information about running Linux processes.

The `/proc` filesystem provides process-specific directories and files that expose information maintained by the kernel.

A monitoring operation can identify a process using its PID, access relevant process information, and display the available details.

The design should also account for situations where a process terminates before its information can be retrieved.

5.6 Process Control Design

Process control involves interacting with the operating system to manage process execution and behavior.

The proposed design considers the following operations:

Process Creation: A new child process can be created using `fork()`.

Program Execution: An existing process can execute another program using an `exec()` family function.

Process Synchronization: A parent process can wait for a child process using `wait()` or a related function.

Signal-Based Control: A signal can be sent to a process using `kill()`.

These mechanisms demonstrate important concepts of operating system process management.

5.7 System Workflow

The overall workflow of the proposed application is:

1. Start the application.
2. Display the available process-management options.
3. Accept the user's selection.
4. Validate the selected operation and its input.
5. Execute the corresponding monitoring or control function.
6. Display the result.
7. Accept another request or exit the application.

This workflow represents the intended command-line interaction model. The final workflow diagram should reflect the actual implemented program.

5.8 Design Considerations

Several factors are considered in the proposed system design.

Simplicity: The interface should be understandable to users learning Linux process management.

Modularity: Monitoring, control, validation, and output responsibilities should be logically separated.

Reliability: The application should handle invalid inputs and unsuccessful system operations without unexpected behavior.

Security: Process-control operations must respect Linux permissions and should not attempt to bypass operating system restrictions.

Maintainability: The code structure should allow future changes without requiring the entire application to be rewritten.

5.9 Advantages of the Proposed Architecture

The proposed architecture offers several advantages.

- Provides a centralized command-line interface for process-related operations.
- Demonstrates Linux process-management concepts through practical interaction.
- Uses operating system interfaces rather than requiring a separate process database.
- Supports a modular approach to software development.
- Can be extended with additional monitoring and control features in the future.

5.10 Summary

This chapter presented the proposed system architecture, logical modules, data flow, and operational workflow of the Linux Process Monitoring and Control System.

The design uses a terminal-based interface to connect user requests with Linux process information and system-level functions. The modular structure provides a foundation for implementing and testing process monitoring and control features.

CHAPTER 6: SYSTEM IMPLEMENTATION

6.1 Introduction

System implementation is the process of converting the proposed system design into an executable software application. It involves writing source code, compiling the program, executing it, and verifying its functionality.

The Linux Process Monitoring and Control System is designed as a command-line application using C/C++ in a Linux environment. It uses Linux process-management interfaces and the `/proc` filesystem to demonstrate important operating system concepts.

6.2 Development Environment Setup

The application is developed in a Linux environment, such as Ubuntu. The development environment includes the GCC compiler, Linux Terminal, and a source-code editor.

The following steps describe the basic setup:

1. Install and configure the Linux operating system.
2. Install the GCC compiler and required development tools.
3. Create the project directory.
4. Write the C/C++ source files.
5. Compile the source code using GCC.
6. Execute the generated program through the terminal.

The development tools provide the environment required to write, compile, and run the application.

6.3 Implementation of Process Monitoring

Process monitoring is an important component of the application. It involves retrieving information about processes currently managed by the Linux operating system.

Linux provides the `/proc` virtual filesystem, which contains information about running processes.

Each process generally has a directory named after its Process ID (PID). The application can access relevant information from these directories.

The important process details include:

- **PID:** Identifies a process uniquely while it exists.
- **PPID:** Identifies the parent process.
- **Process Name:** Represents the name associated with the process.
- **Process State:** Indicates the current state of the process.

The retrieved information can be displayed through the terminal to help users understand process identification and monitoring.

6.4 Implementation of Process Creation

Linux provides the `fork()` system call to create a new process.

When a process calls `fork()`, the operating system creates a child process. The parent and child processes execute independently after the call.

The return value of `fork()` helps distinguish the execution paths:

- A negative value indicates that process creation failed.
- Zero indicates execution in the child process.
- A positive value indicates execution in the parent process and contains the child's PID.
- This mechanism demonstrates the parent-child relationship in Linux process management.

6.5 Implementation of Program Execution

The `exec()` family of functions is used to execute a new program within an existing process.

When an `exec()` function succeeds, the current process image is replaced by the new program. The process retains its PID, but its program code and execution context are replaced.

This mechanism is commonly used together with `fork()`, where a child process is created first and then executes another program.

6.6 Implementation of Process Synchronization

The `wait()` system call allows a parent process to wait for a child process to change state, commonly when the child completes execution.

It helps coordinate the execution of parent and child processes.

Process synchronization is important because it allows the parent process to collect information about the child's termination and manage its execution flow.

6.7 Implementation of Process Control

Linux provides signal-based mechanisms for controlling processes.

The `kill()` system call sends a specified signal to a process using its PID.

Some commonly used signals include:

- **SIGTERM:** Requests process termination.
- **SIGKILL:** Forces process termination.
- **SIGSTOP:** Stops process execution.
- **SIGCONT:** Resumes a stopped process.

The operating system checks permissions before allowing a process to send signals to another process.

6.8 Input Validation and Error Handling

Input validation is necessary to ensure that the application receives appropriate user input.

The system should handle situations such as:

- Invalid process identifiers.
- Unsupported commands.
- Missing input values.
- Processes that no longer exist.
- Insufficient permissions.
- Failed system calls.

Appropriate error messages help users understand the problem and prevent incorrect assumptions about whether an operation succeeded.

6.9 Compilation and Execution

The GCC compiler is used to compile the source code into an executable program.

A typical C compilation command is:

```
gcc main.c -o process_monitor
```

To execute the generated program:

```
./process_monitor
```

For a C++ source file, the compilation command is:

```
g++ main.cpp -o process_monitor
```

The exact command depends on the programming language and filenames used in the project.

6.10 Implementation Challenges

During the development of a Linux process-management application, the following challenges may need to be considered:

1. Understanding Linux process identifiers and process states.
2. Accessing process information through the `/proc` filesystem.
3. Handling errors returned by system calls.
4. Managing parent-child process execution.
5. Handling signals and process permissions.
6. Validating user input and displaying meaningful results.

These challenges highlight the importance of understanding operating system concepts while developing system-level applications.

6.11 Summary

This chapter explained the implementation approach for the Linux Process Monitoring and Control System. It covered the development environment, process monitoring through `/proc`, and important Linux process-management mechanisms such as `fork()`, `exec()`, `wait()`, and `kill()`.

It also described compilation, execution, input validation, and common implementation challenges. These concepts provide the foundation for developing and testing the application in a Linux environment.

CHAPTER 7: TESTING AND RESULTS

7.1 Introduction

Software testing is an important phase of the software development process. It is performed to verify whether an application behaves as expected and to identify errors or unexpected behavior.

The Linux Process Monitoring and Control System requires testing to verify its process-monitoring operations, process-control mechanisms, input validation, and error handling.

Testing helps ensure that the application interacts correctly with the Linux operating system and provides meaningful output to the user.

7.2 Objectives of Testing

The main objectives of testing are:

1. To verify that the application starts and executes correctly.
2. To check whether process information can be retrieved.
3. To verify the identification of processes using PID and PPID.
4. To examine process creation and execution operations.
5. To verify supported process-control operations.
6. To identify errors caused by invalid inputs.
7. To ensure that appropriate messages are displayed when an operation fails.

7.3 Testing Methodology

The application should be tested using different inputs and operating conditions.

The testing process includes:

Functional Testing: Verifies whether each implemented feature performs its intended operation.

Input Validation Testing: Checks how the application handles invalid or incomplete user input.

Error Handling Testing: Verifies whether the application handles unsuccessful operations and displays appropriate messages.

Integration Testing: Checks whether the different modules work together correctly.

7.4 Test Cases

The following test cases can be used to evaluate the application. Execute each test and record the actual result before final submission.

Test Case	Test Description	Expected Result	Actual Result
TC01	Launch the application	Application starts successfully	To be tested
TC02	Retrieve process information	Available process details are displayed	To be tested
TC03	Check PID and PPID	Correct process identifiers are displayed	To be tested
TC04	Create a child process, if implemented	Child process is created successfully	To be tested
TC05	Execute another program, if implemented	Requested program executes successfully	To be tested
TC06	Wait for child process, if implemented	Parent waits for the child process as expected	To be tested
TC07	Send a process signal, if implemented	Signal operation returns an appropriate result	To be tested
TC08	Enter invalid input	Application handles invalid input appropriately	To be tested

7.5 Functional Testing

Functional testing verifies the major operations provided by the application.

The process-monitoring functionality should be checked by retrieving process information from the Linux environment.

The process-control functionality should be checked by performing the supported process-related operations and observing their results.

Each operation should be verified independently to identify errors and confirm that the application behaves according to its intended design.

7.6 Error Handling Testing

Error handling testing evaluates how the application responds to incorrect inputs and unsuccessful system operations.

Important conditions include:

- Entering a PID that does not exist.
- Providing an invalid command.

- Attempting an operation without sufficient permissions.
- Accessing process information after a process has terminated.
- Handling failures returned by Linux system calls.

The application should display understandable error messages instead of terminating unexpectedly.

7.7 Results and Observations

The testing phase should record the actual behavior of the application under different conditions.

The following observations should be documented after execution:

1. Whether the application launches successfully.
2. Whether process information is retrieved correctly.
3. Whether PID and PPID values are displayed correctly.
4. Whether implemented process-control operations work as expected.
5. Whether invalid inputs are handled properly.
6. Whether meaningful output and error messages are displayed.

Insert actual terminal screenshots in this section. Each screenshot should include a figure number, caption, and a short explanation.

Example captions:

- Figure 7.1: Execution of the Linux Process Monitoring Application.
- Figure 7.2: Display of Process Information.
- Figure 7.3: Process Identification Using PID and PPID.
- Figure 7.4: Process Control Operation.
- Figure 7.5: Handling of Invalid Input.

Use only screenshots that demonstrate features actually implemented in your application.

7.8 Limitations Identified During Testing

The application may have certain limitations depending on its implementation and execution environment.

These may include:

- Access restrictions when retrieving information about certain processes.
- Process information changing while monitoring is performed.
- Process-control operations being restricted by Linux permissions.
- Limited monitoring features compared with advanced system-monitoring utilities.
- Dependence on the Linux operating environment.

Only include limitations that are relevant to the actual application.

7.9 Summary

This chapter described the testing methodology and evaluation approach for the Linux Process Monitoring and Control System. It presented functional testing, input validation, error handling, and suggested test cases.

The actual test outcomes and terminal screenshots should be included to demonstrate the behavior of the implemented application. Testing helps identify errors and verify whether the system meets its intended objectives.

CHAPTER 8: CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Introduction

This chapter presents the overall conclusion of the Linux Process Monitoring and Control System. It summarizes the project's objectives, the operating system concepts involved, and the potential improvements that can be introduced in future versions.

8.2 Conclusion

The Linux Process Monitoring and Control System focuses on demonstrating the fundamental concepts of process management in the Linux operating system.

The project provides a command-line-based approach to understanding process information and supported process-control operations. It makes use of Linux facilities such as the `/proc` filesystem and process-related system interfaces.

Through this project, important operating system concepts such as process identification, parent-child relationships, process creation, program execution, synchronization, and signal-based control can be explored.

The development of this project also provides practical exposure to system-level programming using C/C++, Linux, and GCC. It helps connect theoretical knowledge of operating systems with practical programming concepts.

8.3 Learning Outcomes

The project provides the following learning outcomes:

1. Understanding the concept of processes and process management.
2. Learning the difference between processes and programs.
3. Understanding Process ID (PID) and Parent Process ID (PPID).
4. Exploring process information through the Linux `/proc` filesystem.
5. Understanding process creation using `fork()`.
6. Learning program execution using the `exec()` family of functions.
7. Understanding parent-child synchronization using `wait()`.
8. Exploring signal-based process control using `kill()`.
9. Gaining practical experience with Linux system programming.
10. Understanding the importance of input validation and error handling.

8.4 Future Enhancements

The current project can be extended with additional features to improve its functionality and usability.

8.4.1 Graphical User Interface

A graphical user interface can be introduced to make process monitoring easier for users who are not comfortable with command-line applications.

8.4.2 CPU and Memory Monitoring

Future versions can include CPU utilization and memory consumption information for individual processes.

8.4.3 Real-Time Process Monitoring

The system can be enhanced to periodically refresh process information and provide continuously updated monitoring.

8.4.4 Process Priority Management

Additional functionality can be introduced to view and manage process priorities using Linux scheduling-related utilities and interfaces, subject to system permissions.

8.4.5 Process Activity Logging

A logging mechanism can be added to maintain records of process-related operations, timestamps, and execution results.

8.4.6 Alerts and Notifications

The system can be extended to notify users about selected process events, such as process termination or unexpected execution failures.

8.4.7 Advanced Process Filtering

Future versions can support filtering and searching processes using attributes such as PID, process name, and process state.

8.5 Final Remarks

The Linux Process Monitoring and Control System demonstrates the importance of process management in modern operating systems.

By exploring Linux process information and system-level interfaces, the project provides a practical foundation for understanding how processes are created, executed, monitored, and controlled.

The proposed enhancements provide opportunities to expand the application into a more comprehensive process-monitoring utility in the future.

8.6 Summary

This chapter summarized the project's purpose, learning outcomes, and possible future improvements. The project connects theoretical operating system concepts with practical Linux programming and provides a foundation for further exploration of process management.

CHAPTER 9: REFERENCES AND BIBLIOGRAPHY

9.1 Introduction

This chapter lists the technical documentation, programming references, and learning resources relevant to the development and understanding of the Linux Process Monitoring and Control System.

The following references cover Linux process management, system programming, C/C++ development, and operating system concepts.

9.2 References

1. Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, *Operating System Concepts*, 10th Edition, Wiley.
2. W. Richard Stevens and Stephen A. Rago, *Advanced Programming in the UNIX Environment*, 3rd Edition, Addison-Wesley.
3. Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, 2nd Edition, Prentice Hall.
4. Linux Manual Pages – Process Management Functions.

- `fork(2)`
- `exec(3)`
- `wait(2)`
- `kill(2)`
- `signal(7)`

5. Linux Manual Pages – `/proc` Filesystem Documentation.

- `proc(5)`

6. GNU Compiler Collection Documentation.

- https://gcc.gnu.org/onlinedocs/

7. Linux Kernel Documentation.

- https://docs.kernel.org/

8. Ubuntu Official Documentation.

- https://documentation.ubuntu.com/

9.3 Online Resources

The following online resources can be referred to for additional technical information:

- Linux Manual Pages: https://man7.org/linux/man-pages/
- GNU Compiler Collection: https://gcc.gnu.org/

- Linux Kernel Documentation: https://docs.kernel.org/
- Ubuntu Documentation: https://documentation.ubuntu.com/

9.4 Summary

The references listed in this chapter provide supporting information about operating system concepts, Linux process management, system calls, programming languages, and development tools.

These resources serve as technical references for understanding the concepts associated with the Linux Process Monitoring and Control System.